Testing Workshop

ICCS Summer School 2025

Dominic Orchard and Isaac Akanho

Validation

Did we implement the right equations?

vs

Verification

Did we implement the equations right?

Challenge

Telling these two apart when results are not as expected

Terminology: what does "*verified*" mean?

Verification wrt. a specification

i.e. check(implementation, specification)

The value of a specification is what we make of it;
it depends on our goals, values, and ***which risks we wish to mitigate against***

Validation as a proxy for verification

- Can we get a stable run (over decades)?
- Is it plausible from physics perspective?
- Do hindcasts reproduce observational record?

If the science is relatively settled, then points to bugs not invalidity

Problem: error localisation is poor!

Our focus: Testing

- Implementation language = specification language
- Test subset of inputs / program behaviours

  Does not show “absence of bugs” (Dijkstra); may fail to expose unconceived-of bugs
- Key principles:
  - automate testing so that it is easy to deploy
  - speed up development time by reducing bug hunting time
  - increase confidence in code
  - enable safe(r) refactoring (*want semantics preservation*)

Plan

- We will use `python` and `pytest`
- Part 0 - Smoke tests
- Part 1 - Unit tests
- Part 2 - Integration tests
- Part 3 - Property-based tests
- Live demos throughout + exercises based on a 0D Energy Balance model
  https://github.com/Cambridge-ICCS/testing-workshop

Schedule

Session 1 - 1h

- Explaining concepts about unit testing including
  - Parameterised tests
  - Fixtures
  - Negative tests
  - Approximation and floating point
  - TDD
  - Code coverage

Session 2 - 1h30

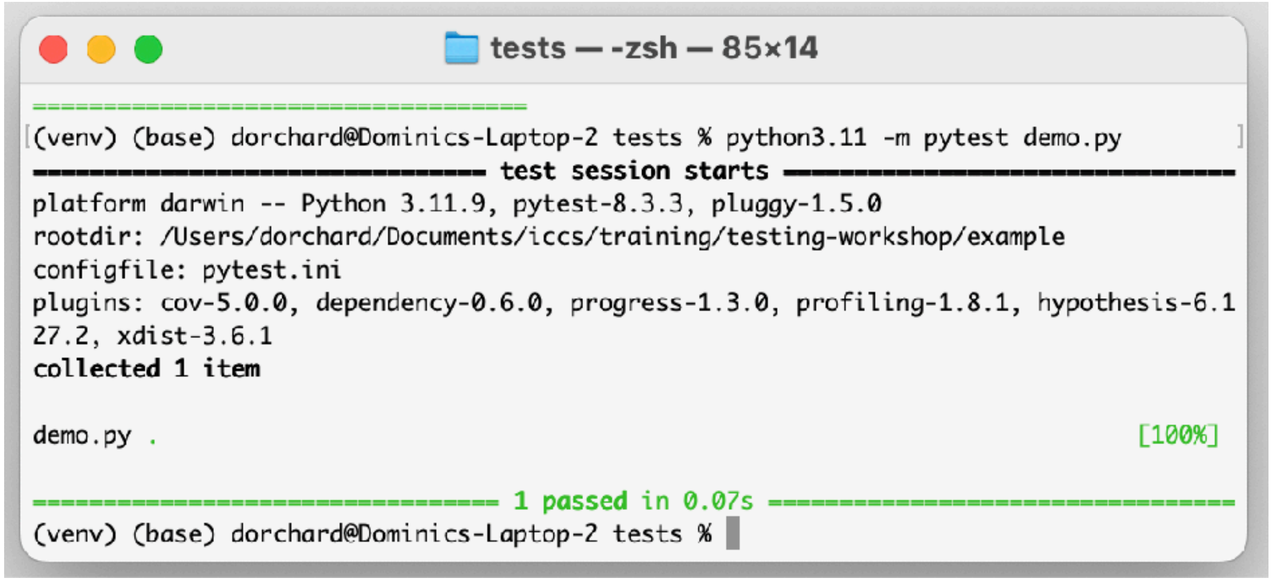
- 20 minute unit test exercises
- 40 minutes lecture
  - Integration and end-to-end tests
  - Property-based testing
- 30 minutes exercises (property-based test exercises)

pytest basics

- Put tests as .py files in a `tests/` directory
- Each test group is a function with prefix `test_`
- Use `assert` keyword with a boolean expression

tests/demo.py

```
def func(x):
    return x + 1

def test_answer():
    assert func(3) == 4
```



`assert` statement anatomy

```
assert boolean-expr
```

```
assert boolean-expr, string-expr
```

Meaning: if `boolean-expr` false, show `string-expr` in the output

Part 0 - Smoke tests

What is a smoke test?

🔥 *Did it catch fire?*

- Was the program able to “start” (initialise) without throwing an exception?
  - *Might include loading configuration files or data sets*

```
def test_config_loading():
    # Check that init does not fail
    energy_balance.init()
    # and that it initializes a global variable
    assert energy_balance.solar_constant is not None, "Solar constant should be initialized"
```

Part I - Unit tests

Unit test basics (1)

- Applied to smaller “units” of code…
- Run code, then compare actual results against expected results
  Conceptually….

$$\text{square} : \text{float} \rightarrow \text{float}$$

$$\text{square}(2) \equiv 4$$

Unit test basics (2)

- Try various values *systematically:*
  - with simple / base cases

$$\text{square}(0) \equiv 0$$
$$\text{square}(1) \equiv 1$$

-with more complex cases

$$\text{square}(4) \equiv 16$$
$$\text{square}(-3) \equiv 9$$

- with corner cases (*requires more creative thought sometimes)*

$$\text{square}(\text{NaN})???$$

Unit test demo

tests/test_helpers.py

```
from helpers import square
from math import nan, isnan

def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```

(NaN ≠ NaN use `math.isnan` instead)

Floating point considerations…

- NaN ≠ NaN use `math.isnan` instead
- Floating-point rounding error can be a pain,
  - `pytest.approx`
    - e.g., `pytest.approx(0.614618, abs=1e-4)`
      - `rel` – Relative tolerance (good when similar scale)
      - `abs` - Absolute tolerance (good when different scales)
  - OR `np.isclose / np.allclose`

Test *parameterization*

Reduce repetition by defining a single test over different inputs

tests/test_helpers.py

```python
from helpers import square
from math import nan, isnan

def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```

tests/test_helpers_param.py

```python
from helpers import square
from math import nan, isnan
import pytest

def test_square_nan():
    assert isnan(square(nan))

@pytest.mark.parametrize("value, expected", [
    (0, 0),
    (1, 1),
    (4, 16),
    (-1, 1),
])
def test_square_parametrized(value, expected):
    assert square(value) == expected
```

Test fixtures

Provide context or content for tests

tests/conftest.py

```
import energy_balance
import pytest

@pytest.fixture
def setup():
    energy_balance.init()
```

Define a fixture

tests/test_energy_balance.py

```
import energy_balance

def test_energy_in_albedo_max(setup):
    assert energy_balance.energy_in(1) == 0
```

Use it in tests that need it

Negative tests

“*We expect this to fail with an exception*”

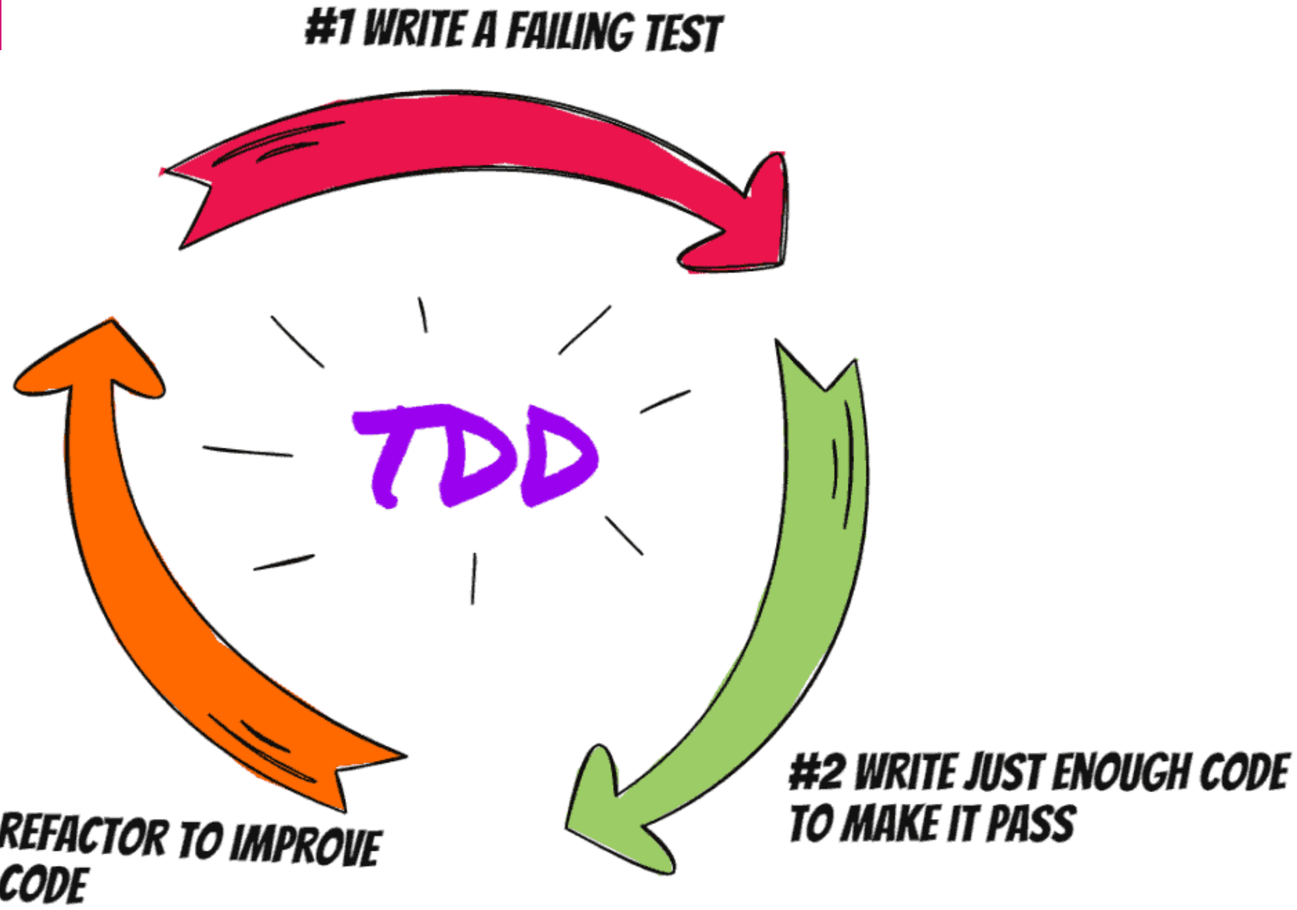
```
def test_energy_in_invalid(setup):
    albedo = -0.1
    with pytest.raises(ValueError):
        energy_balance.energy_in(albedo)
```

Unit test basics (3)

- **General principle:** try to cover a representative sample of input space.
- For data structures:
    - Empty data structure
    - Small data structures
    - All constructor shapes (i.e., all small sizes of binary tree)
    - Very large?

Test-Driven Development (TDD)

- **Specify first, implement second**
- TDD cycle is quite fine grained
  - #1 could be repeated first
- Encourage more modular design
- ‘Correctness first’ mindset



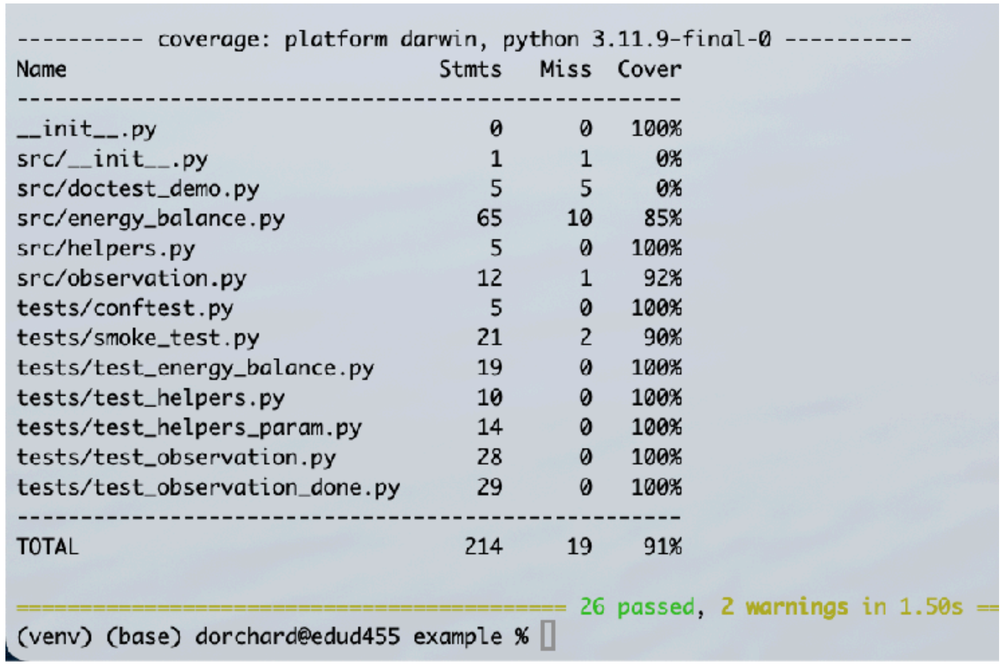
https://methodpoet.com/test-driven-development-best-practices/ (Kristijan Kralj)

Coverage

What % of *control-flow paths* have a test?

`pytest test-path --cov code-path`

e.g. `pytest tests --cov .`



```
---------- coverage: platform darwin, python 3.11.9-final-0 ----------
Name                                 Stmts   Miss  Cover
--------------------------------------------------------
__init__.py                              0      0   100%
src/__init__.py                          1      1     0%
src/doctest_demo.py                      5      5     0%
src/energy_balance.py                   65     10    85%
src/helpers.py                           5      0   100%
src/observation.py                      12      1    92%
tests/conftest.py                        5      0   100%
tests/smoke_test.py                     21      2    90%
tests/test_energy_balance.py            19      0   100%
tests/test_helpers.py                   10      0   100%
tests/test_helpers_param.py             14      0   100%
tests/test_observation.py               28      0   100%
tests/test_observation_done.py          29      0   100%
--------------------------------------------------------
TOTAL                                  214     19    91%

========================================= 26 passed, 2 warnings in 1.50s ==
(venv) (base) dorchard@edud455 example %
```

To find out which control-flow paths (per line) are not covered:

`pytest test-path --cov code-path --cov-report term-missing`

`pytest test-path --cov code-path --cov-report html`

Part II - Integration tests

“End-to-end”

What is a Component?

- A self-contained slice of logic
- May embed smaller components inside it
- An example of a minimal component could be a function which does not call any other function

```
def minus_one(x: int) → int:
    return x - 1
```

What is Integration Testing?

- Tests two or more components, modules or (sub)systems working together
- Focuses on the connections rather than isolated logic

What is Integration Testing?

- Example chain: A → B → C – verify that data flows smoothly through the functions

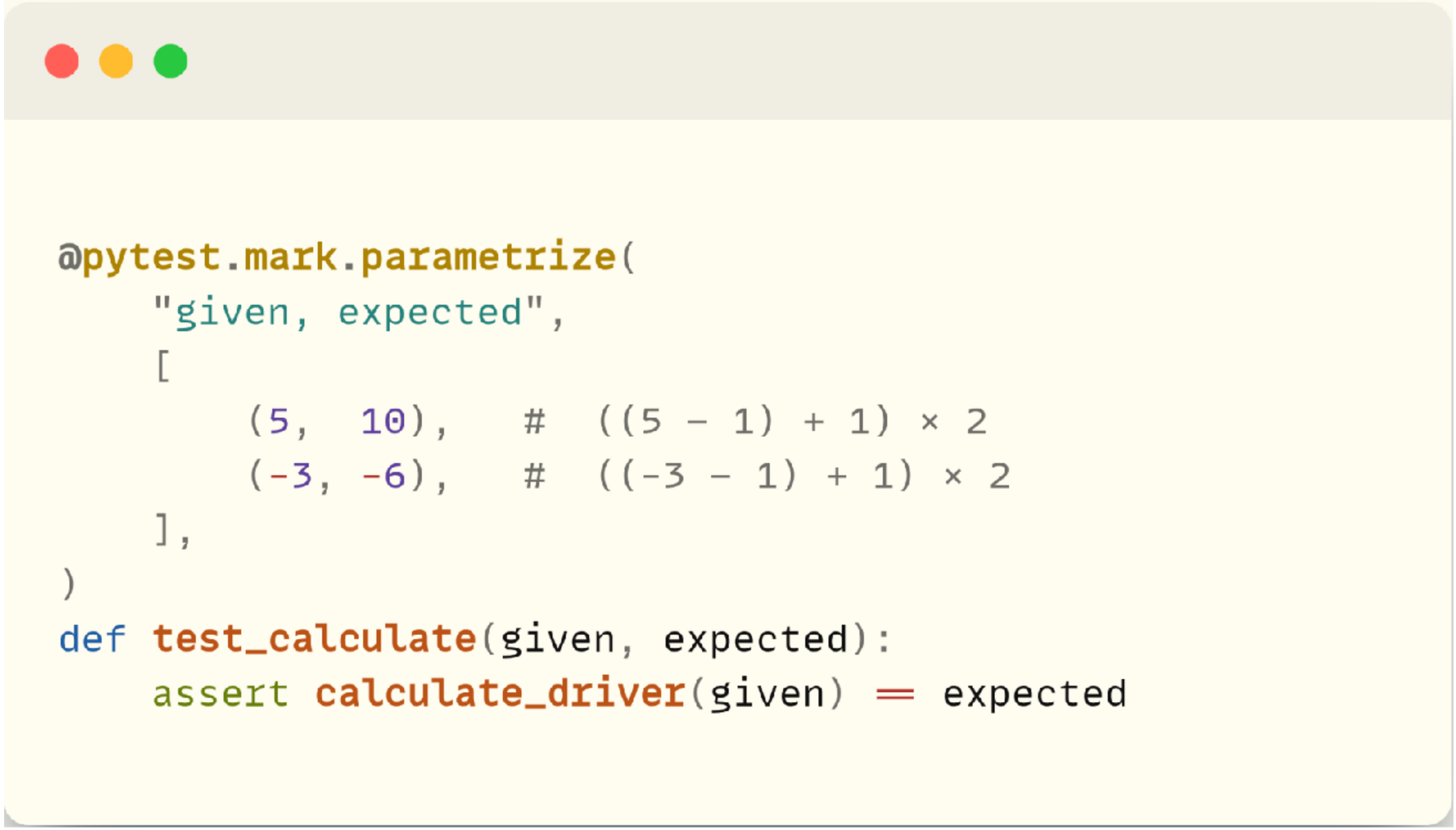
```python
def minus_one(x: int) -> int:
    return x - 1

def add_one(x: int) -> int:
    return x + 1

def double(x: int) -> int:
    return x * 2
```

What is Integration Testing?

```python
def calculate_driver(x: int) -> int:
    """
    Runs: minus_one → add_one → double
    """
    return double(add_one(minus_one(x)))
```

This is called a driver function

What is Integration Testing?



```python
@pytest.mark.parametrize(
    "given, expected",
    [
        (5,  10),   #  ((5 - 1) + 1) × 2
        (-3, -6),   #  ((-3 - 1) + 1) × 2
    ],
)
def test_calculate(given, expected):
    assert calculate_driver(given) == expected
```

Stubbing Dependencies

- Replace real inputs (data, services, functions)
- Speeds up tests and isolates the behaviour under scrutiny

Stubbing Dependencies

```python
def calculate_temperature_change(heat_flux, layer_depth, initial_temp):
    """
    Args:
        heat_flux: W/m² (positive = heat into ocean)
        layer_depth: m
        initial_temp: °C
    Returns:
        new temperature (°C)
    """
    mass = DENSITY_WATER * layer_depth  # kg/m²
    delta_temp = heat_flux / (mass * SPECIFIC_HEAT)
    return initial_temp + delta_temp
```

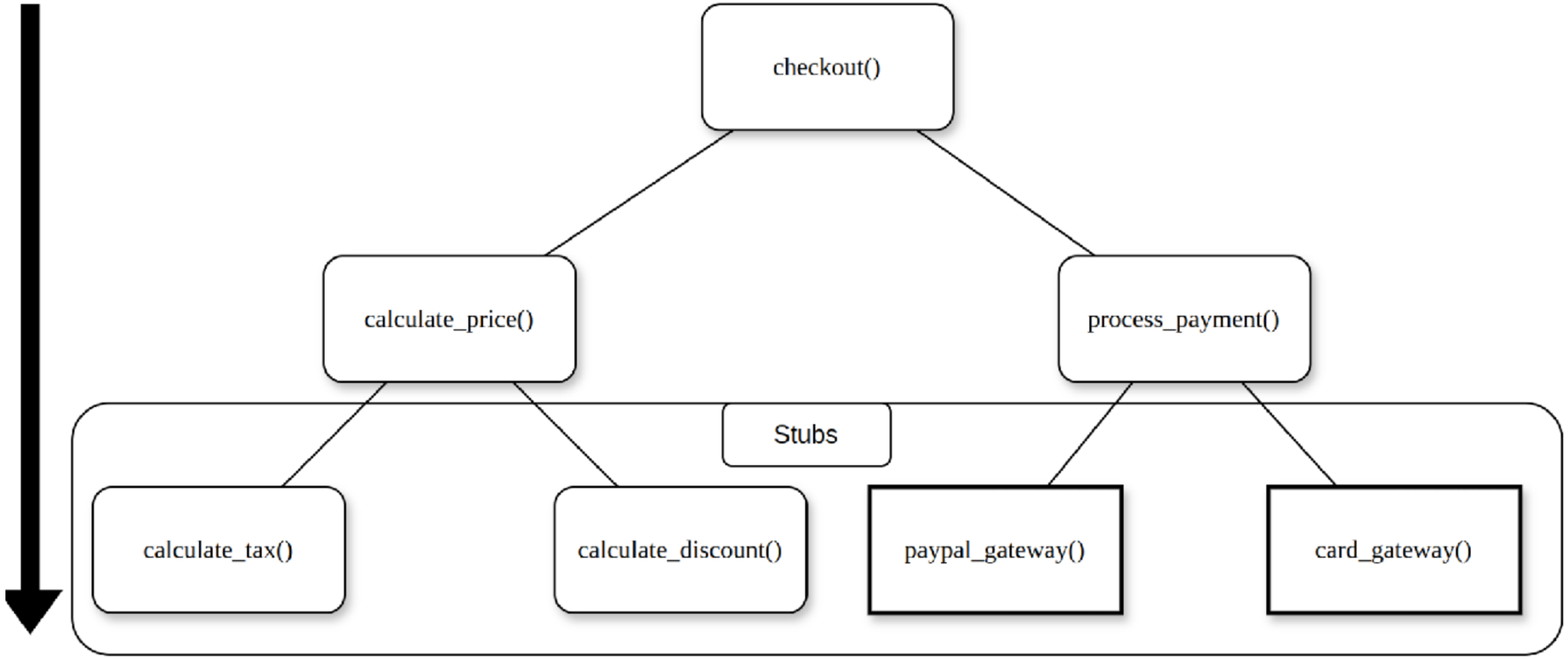
Stubbing Dependencies

```
def stub_calculate_temperature_change(heat_flux, layer_depth, initial_temp):
    return 5
```

Top-Down Testing

- Start with higher-level components first
- Incrementally add and test each lower layer until you reach the leaf of the tree
- Good for validating broad behaviour early

Top-Down Testing



Bottom-Up Testing

- Begin with the lowest-level components
- Build upward, testing each new layer as you integrate it
- Helpful when low-level code is more complex or you are doing a refactor

Bottom-Up Testing

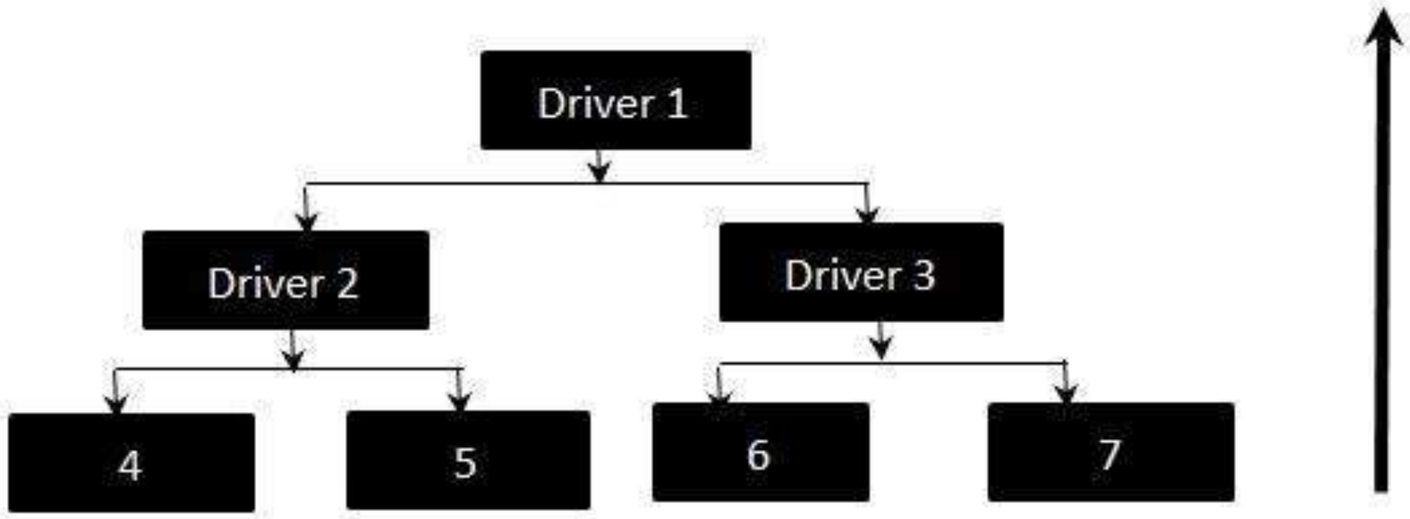


Image Credit: microsoft.github.io/code-with-engineering-playbook

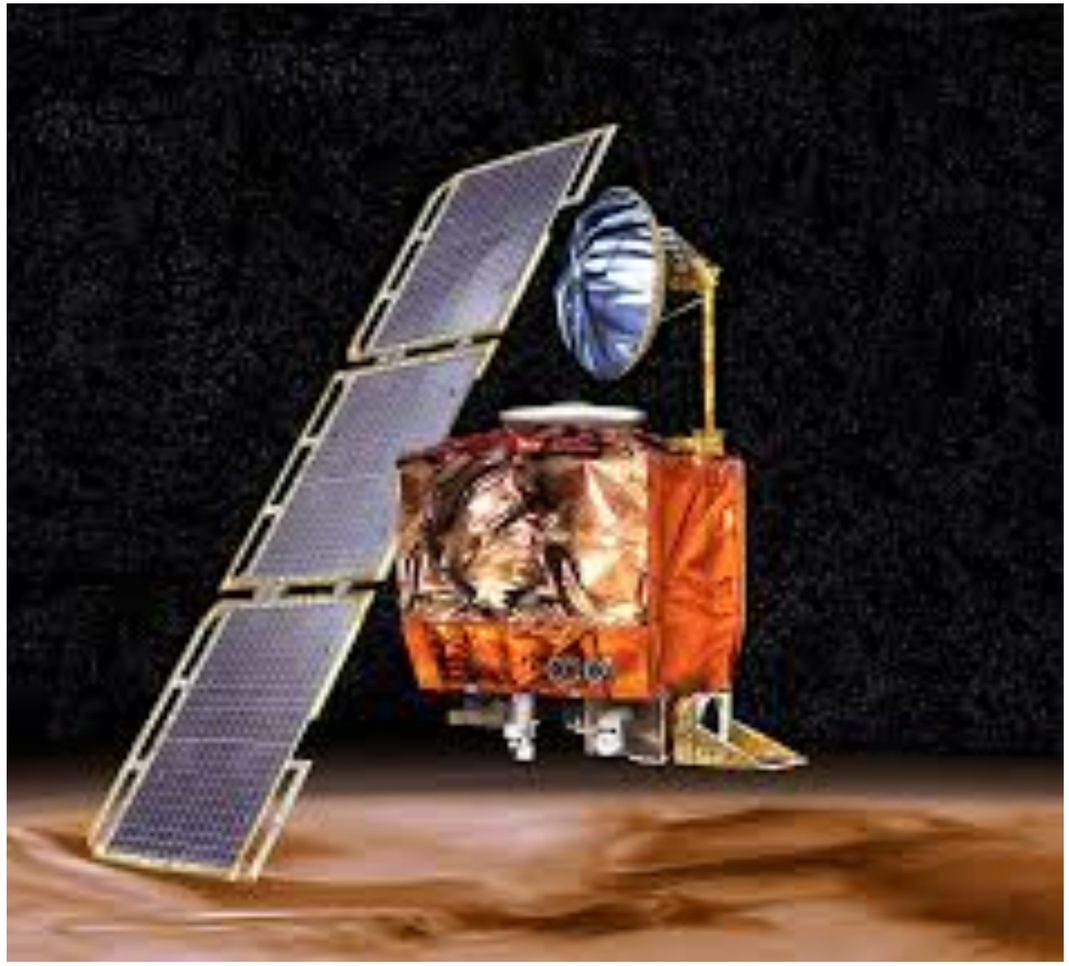
Choosing an Approach

- No single style fits every project
- Pick the approach that suits your codebase, purpose, and time constraints
- Be pragmatic – mix and match when it helps

Interfaces: Where Things Break

- Bugs often hide in inconsistent expectations between components
- Common mismatches:
  - Units (km vs m)
  - Time steps (daily vs hourly)
  - Percentage vs decimal
  - Unexpected NaNs in data
  - Wrong data types
  - Memory leaks

Testing: Space



- After 10 months of travelling, the mars orbiter burned up
- Led to a loss of $125 million, years of work
- Caused by calculations with mismatched units

Quick Self-Checks

- What should I prioritise for integration?
  - High risk interfaces
  - Complex combinations of components
  - Components which integrate components you plan to change

End-to-End (E2E) Testing

- Run the entire system from start to finish
- Example: a complete data-analysis or climate-forecast pipeline
- Typically cover the main program flows and edge paths

End-to-End (E2E) Testing

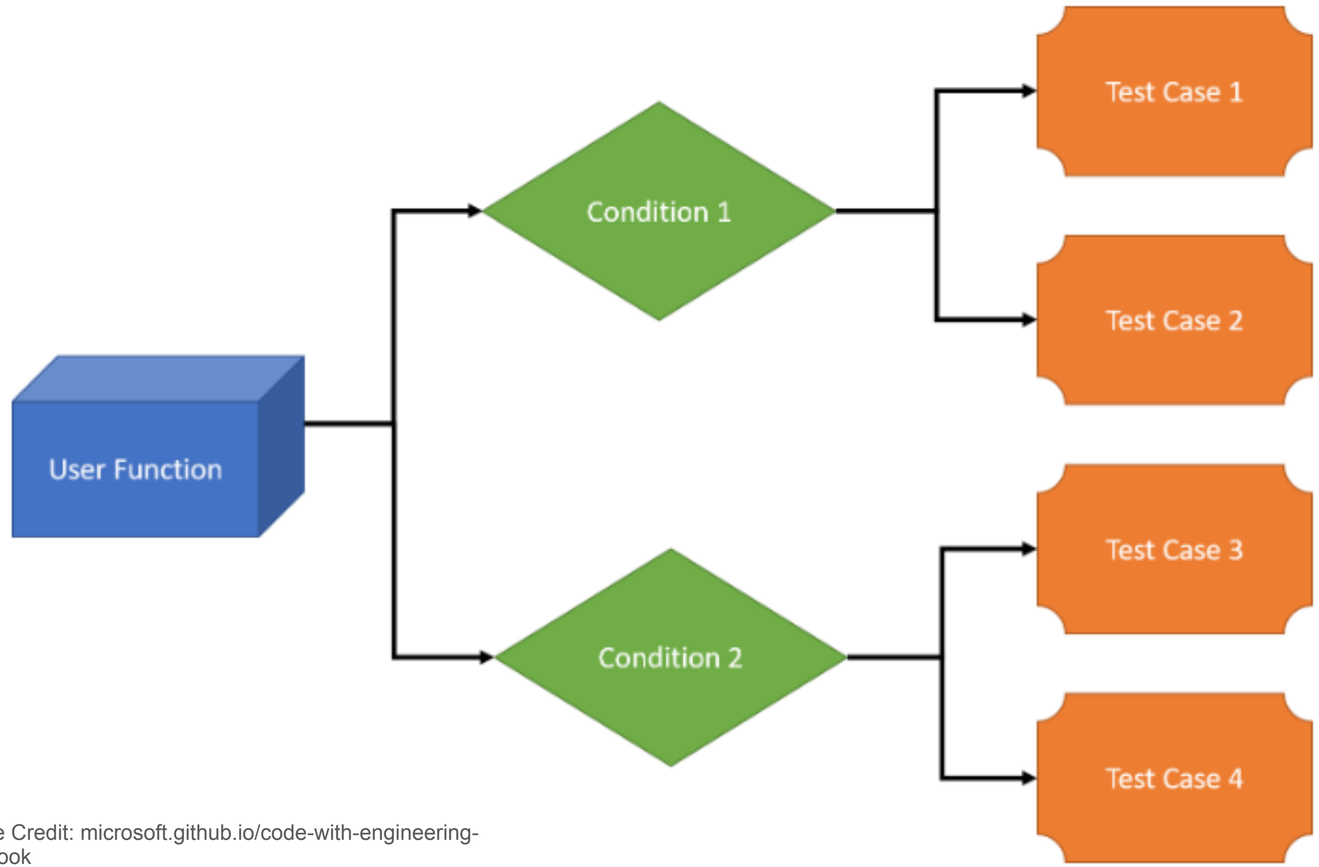


Image Credit: microsoft.github.io/code-with-engineering-playbook

End-to-End (E2E) Testing

- Pros
  - Confirms that everything works together
- Cons
  - Slowest tests
  - Hardest for localising bugs

End-to-End (E2E) Testing

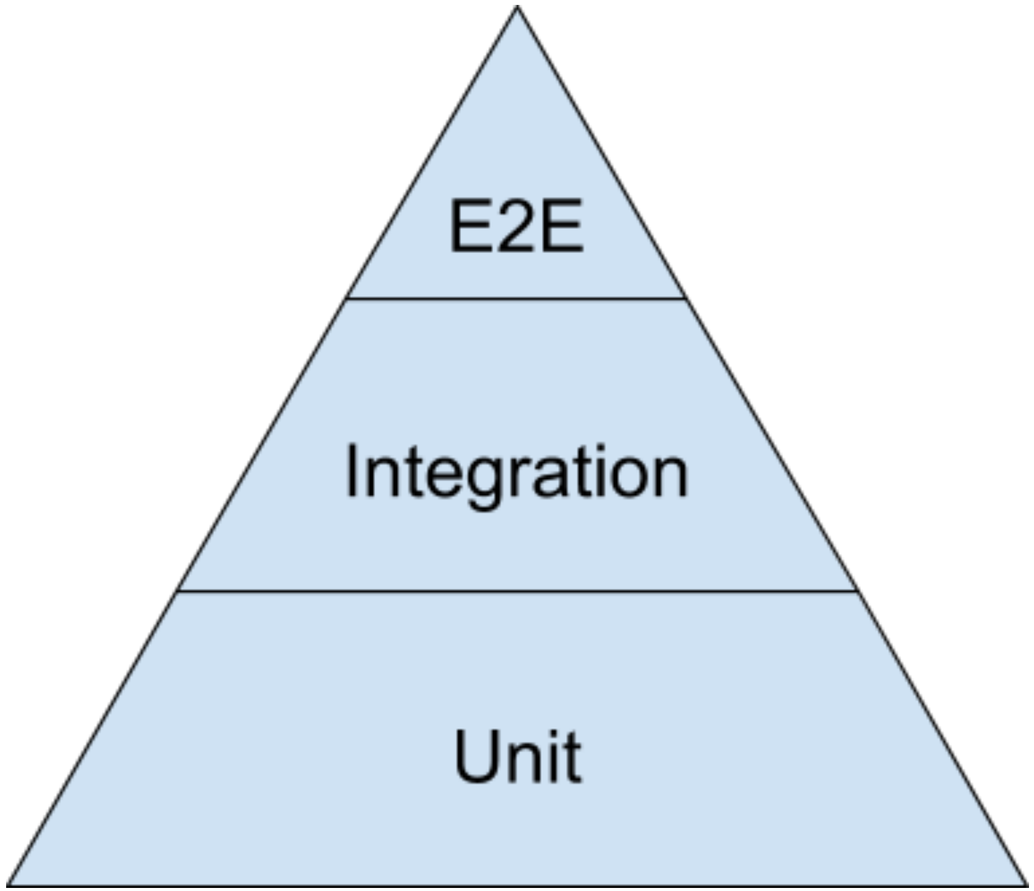


Image Credit: https://testing.googleblog.com/2015/04/just-say-no-to-more-end-to-end-tests.html

Summary

- Integrate early, test often, stub when appropriate
- Choose top-down, bottom-up, or a hybrid based on context
- Keep an eye on interfaces – common trip up

Part III - Property-based tests

Does integration + unit testing scale?

```
def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```

A small subset of the input space

Thinking of corner cases is hard
Writing lots of cases is time consuming

Can we automate?

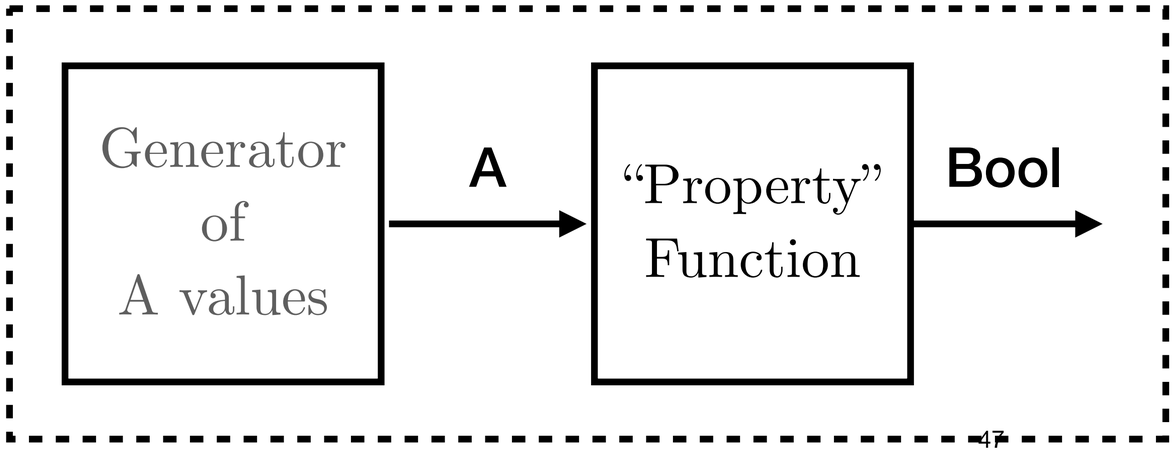
Property-based testing basis

- Tests based on a general property over broad input space

```
def property(x : str) -> bool:
  return (reverse(reverse(x)) == x)
```

"Generator" for `str` systematically specialises / randomises inputs, finding counterexamples

see QuickCheck (Haskell), hypothesis (Python)

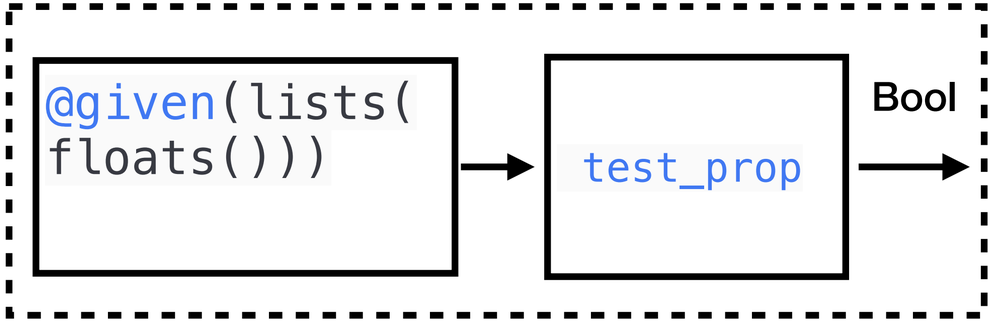


Framework runs this N times
(usually N=100)

False outputs provide counter-examples
(these are also "shrunk")

Python hypothesis library; strategies

- Generators called “strategies”




```
from hypothesis import given
from hypothesis.strategies import floats, lists

@given(lists(floats()))
def test_prop(input_data):
  …
```

- Automatically generates 100 inputs of the given form; runs `test_prop` with each
- Strategies can be configured to choose a subset of the domain, e.g.,

```
floats(allow_infinity=False, allow_nan=False)
```

Custom / composite strategies

Example: two random lists of the same length

- Define a function, expecting a function as input which ‘draws out’ values from other strategies (draw)

Defines a strategy that is built out of potentially arbitrarily many other strategies.

@composite provides a callable `draw` to dynamically draw a value from any strategy

```
@composite
# Generate two lists of the same length
def same_len_float_lists(draw):
    list1 = draw(lists(floats()))
    list2 = draw(lists(floats(),
        min_size=len(list1),
        max_size=len(list1)
    ))
    return (list1, list2)

@given(same_len_float_lists())
def test_msqerror2(input_data):
  model_data, obs_data = input_data
…
```

Diverse Inputs + Strong Properties $\Longrightarrow$ Strong(er) Tests

How to create strong properties?

Defining useful properties

“Avoid replicating your code in your tests.” [1]

1. Postconditions - Relation between return values/arguments; what is true after?
   e.g., for reverse/sort, every element in input list is also in output (and vice versa)
   *[**but on its own this is not sufficient**]*

2. Validity Testing - Every operation should return valid results
   Define a notion of validity as a boolean-valued function

3. Algebraic properties - Involution? Commutativity? Associativity? Unitality?

4. Metamorphic properties - Related calls return related results
   Relevant in science: invariance / symmetries
   Conservation of energy/momentum/mass etc.

[1] How to Specify It!: A Guide to Writing Properties of Pure Functions - Hughes 2019

Final thoughts

- Powerful technique
- Combine with other approaches
- Custom generators+shrinkers (further work)
- Frameworks exist in most languages

Property Testing for Ocean Models. Can We Specify It?

Deepak A. Cherian
Golden, Colorado
deepak@cherian.net

I take inspiration from the property-testing literature, particularly the work of Prof. John Hughes [17], and explore how such ideas might be applied to numerical models of the ocean. Specifically, I ask whether geophysical fluid dynamics (GFD) theory, expressed as property tests, might used to address the oracle problem of testing the correctness of ocean models. I propose that a number of simple idealized GFD problems can be framed as property tests. These examples clearly illustrate how physics naturally lends itself to specifying property tests. Which of these proposed tests might be most feasible and useful, remains to be seen.

1 Introduction

1.1 Background

Simulated possible projections of the Earth's climate by Earth System Models are an important tool in understanding, adapting to, and mitigating the effects of climate change. Such models bring together a number of individual component models (e.g. the atmosphere, ocean, land, rivers, and more) coupled to each other through modeled interactions at the interfaces between these components (for example, the sea surface). The complexity of each individual component is vast, the complexity of a full Earth System Model is staggering (Figure 1).

Consider the ocean. The ocean is a thin shell of fluid on the Earth[1]. The ocean is "stratified", meaning the density of seawater varies with depth so that lighter fluid overlays denser fluid. Commonly, warm and fresh (low salinity) water overlays colder, saltier water. The equations that describe the motion of the ocean are the Navier-Stokes equations specialized for rotating fluids. These equations express the

[1] compare its depth of ~5 km to the radius of the earth (~6400 km)

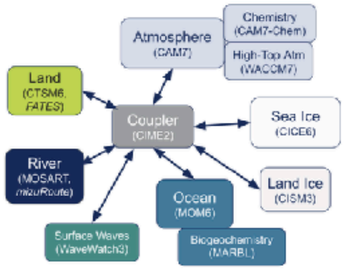


Figure 1: Schematic of the Community Earth System Model version 3 (CESM3) [20].

Submitted to:
VSS 2025

Thanks- and happy testing!

https://iccs.cam.ac.uk